

1. MEAN MEDIAN MODE

```
import statistics as stats

data = list(map(int, input("Enter numbers separated by spaces: ").split()))

(or)

data=[11,12,15,17,13,16,14,15,11,10]

mean = stats.mean(data)

median = stats.median(data)

mode = stats.mode(data)

variance = stats.variance(data)

std_deviation = stats.stdev(data)

print(f'Mean: {mean}\nMedian: {median}\nMode: {mode}\nVariance: {variance}\nStandard
Deviation: {std_deviation}')
```

2. LINEAR REGRESSION

```
import numpy as np

import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split

from sklearn.linear_model import LinearRegression

from sklearn.metrics import mean_squared_error, r2_score

x = 2 * np.random.rand(100, 1)

y = 4 + 3 * x + np.random.randn(100, 1)

x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.2, random_state=0)

model = LinearRegression().fit(x_train, y_train)

y_pred = model.predict(x_test)

print(f'Coefficients: {model.coef_[0][0]:.2f}')

print(f'Intercept: {model.intercept_[0]:.2f}')

print(f'Mean Squared Error: {mean_squared_error(y_test, y_pred):.2f}')

print(f'Coefficient of Determination R^2: {r2_score(y_test, y_pred):.2f}')

plt.scatter(x_test, y_test, color='black', label='Actual data')
```

```
plt.plot(x_test, y_pred, color='blue', label='Regression Line')  
plt.legend(), plt.show()
```

3. MULTIPLE REGRESSION

```
import numpy as np, matplotlib.pyplot as plt  
  
from sklearn.model_selection import train_test_split  
from sklearn.linear_model import LinearRegression  
from sklearn.metrics import mean_squared_error, r2_score  
  
X = np.hstack([np.random.rand(100, 1) for _ in range(2)])  
y = 3 * X[:, 0] + 5 * X[:, 1] + np.random.randn(100)  
  
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=0)  
  
model = LinearRegression().fit(X_train, y_train)  
  
y_pred = model.predict(X_test)  
  
print(f'Coefficients: {model.coef_}, Intercept: {model.intercept_}')  
  
print(f'Mean Squared Error: {mean_squared_error(y_test, y_pred):.2f}, Coefficient of determination  
R^2: {r2_score(y_test, y_pred):.2f}')  
  
fig, axes = plt.subplots(1, 2, figsize=(10, 4))  
  
for i, ax in enumerate(axes):  
    ax.scatter(X_test[:, i], y_test, color='black', label='Actual')  
    ax.scatter(X_test[:, i], y_pred, color='blue', label='Predicted')  
    ax.set(title=f'X{i+1} vs y', xlabel=f'X{i+1}', ylabel='y')  
    ax.legend()  
  
plt.tight_layout()  
plt.show()
```

4. LOGISTIC REGRESSION USING SKLEARN

```
import numpy as np, matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split

from sklearn.linear_model import LogisticRegression

from sklearn.metrics import (accuracy_score, confusion_matrix, classification_report, roc_curve,
                             auc)

X, y = np.random.rand(100, 2), (np.sum(np.random.rand(100, 2), axis=1) > 1).astype(int)

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=0)

model = LogisticRegression().fit(X_train, y_train)

y_pred, y_proba = model.predict(X_test), model.predict_proba(X_test)[:, 1]

print(f'Accuracy: {accuracy_score(y_test, y_pred):.2f}')

print(confusion_matrix(y_test, y_pred), classification_report(y_test, y_pred))

fpr, tpr, _ = roc_curve(y_test, y_proba)

plt.plot(fpr, tpr, label=f'ROC (AUC = {auc(fpr, tpr):.2f})', color='orange')

plt.plot([0, 1], [0, 1], 'r--', plt.xlabel('FPR'), plt.ylabel('TPR'))

plt.legend()

plt.show()
```

5. BINARY CLASSIFICATION

```
from sklearn.model_selection import train_test_split

from sklearn.linear_model import LogisticRegression

from sklearn.datasets import load_iris

from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

X, y = load_iris(return_X_y=True)

X, y = X[y != 2], y[y != 2]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)

model = LogisticRegression().fit(X_train, y_train)

y_pred = model.predict(X_test)

print(f'Acc: {accuracy_score(y_test, y_pred):.2f}')

print('Confusion Matrix:\n', confusion_matrix(y_test, y_pred))

print('Classification Report:\n', classification_report(y_test, y_pred))
```

6. KKN AND NAIVE BAYES ALGORITHM

```
from sklearn.datasets import make_classification

from sklearn.model_selection import train_test_split

from sklearn.neighbors import KNeighborsClassifier

from sklearn.naive_bayes import GaussianNB

from sklearn.metrics import accuracy_score, classification_report

X, y = make_classification(n_samples=1000, n_features=20, n_classes=2, random_state=12345)

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=12345)

for model in [KNeighborsClassifier(), GaussianNB()]:

    model.fit(X_train, y_train)

    preds = model.predict(X_test)

    print(f'{model.__class__.__name__} Accuracy: {accuracy_score(y_test, preds):.2f}')

    print(f'{model.__class__.__name__} Classification Report:\n{classification_report(y_test,
preds)}\n')
```

7. DECISION TREE FOR CLASSIFICATION USING SKLEARN AND ITS PARAMETER TUNING

```
import pandas as pd

from sklearn.model_selection import train_test_split

from sklearn.tree import DecisionTreeClassifier, plot_tree

import matplotlib.pyplot as plt

data = pd.read_csv('https://archive.ics.uci.edu/ml/machine-learning-databases/balance-
scale/balance-scale.data', header=None)

print("Dataset Length:", len(data))

print("Dataset Shape:", data.shape)

print("Dataset:\n", data.head())

X, y = data.values[:, 1:5], data.values[:, 0]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=100)

clf_gini = DecisionTreeClassifier(criterion="gini", random_state=100, max_depth=3,
min_samples_leaf=5).fit(X_train, y_train)
```

```

clf_entropy = DecisionTreeClassifier(criterion="entropy", random_state=100, max_depth=3,
min_samples_leaf=5).fit(X_train, y_train)

for clf, name in zip([clf_gini, clf_entropy], ["Gini", "Entropy"]):

    plt.figure(figsize=(15, 10))

    plot_tree(clf, filled=True, feature_names=['X1', 'X2', 'X3', 'X4'], class_names=['L', 'B', 'R'],
rounded=True)

    plt.title(f'Decision Tree - {name}')

    plt.show()

```

8. K-MEANS ALGORITHM

```

import numpy as np

import matplotlib.pyplot as plt

data = np.array([[1, 2], [1, 4], [1, 0], [10, 2], [10, 4], [10, 0]])

plt.scatter(data[:, 0], data[:, 1], s=100); plt.title('Dataset'); plt.show()

def k_means(data, k, max_iters=100):

    np.random.seed(42)

    centroids = data[np.random.choice(data.shape[0], k, replace=False)]

    for _ in range(max_iters):

        labels = np.argmin(np.linalg.norm(data[:, np.newaxis] - centroids, axis=2), axis=1)

        new_centroids = np.array([data[labels == i].mean(axis=0) for i in range(k)])

        if np.all(centroids == new_centroids): break

        centroids = new_centroids

    return centroids, labels

centroids, labels = k_means(data, 2)

plt.scatter(data[:, 0], data[:, 1], c=labels, s=100, cmap='viridis')

plt.scatter(centroids[:, 0], centroids[:, 1], c='red', s=200, marker='X')

plt.title('K-means Clustering')

plt.show()

```

9. IMAGE CLASSIFIER USING CNN IN TENSORFLOW/KERAS

```
import tensorflow as tf

from tensorflow.keras import datasets, layers, models

import matplotlib.pyplot as plt

(train_images, train_labels), (test_images, test_labels) = datasets.cifar10.load_data()

train_images, test_images = train_images / 255.0, test_images / 255.0

model = models.Sequential([

    layers.Conv2D(32, (3, 3), activation='relu', input_shape=(32, 32, 3)),

    layers.MaxPooling2D(),

    layers.Conv2D(64, (3, 3), activation='relu'),

    layers.Flatten(),

    layers.Dense(64, activation='relu'),

    layers.Dense(10, activation='softmax')

])

model.compile(optimizer='adam', loss='sparse_categorical_crossentropy', metrics=['accuracy'])

history = model.fit(train_images, train_labels, epochs=10, validation_data=(test_images,
test_labels))

print(f"Test accuracy: {model.evaluate(test_images, test_labels, verbose=2)[1]:.4f}")

plt.figure(figsize=(10, 4))

for i, key in enumerate(['accuracy', 'loss']):

    plt.subplot(1, 2, i + 1)

    plt.plot(history.history[key], label='Train ' + key)

    plt.plot(history.history['val_' + key], label='Val ' + key)

    plt.xlabel('Epoch')

    plt.ylabel(key.capitalize())

    plt.legend()

plt.show()

predicted_class=tf.argmax(tf.keras.models.load_model('cifar10_cnn_model.h5').predict(test_images
[0:1]))

print(f"Predicted class: {predicted_class.numpy()[0]}")
```

10.AUTOENCODER IN TENSORFLOW/KERAS

```
import tensorflow as tf

import matplotlib.pyplot as plt

(X_train, _), (X_test, _) = tf.keras.datasets.mnist.load_data()

X_train, X_test = [x.reshape((-1, 784)) / 255.0 for x in (X_train, X_test)]

autoencoder = tf.keras.models.Sequential([

    tf.keras.layers.Dense(32, activation='relu', input_shape=(784,)),

    tf.keras.layers.Dense(784, activation='sigmoid')

])

autoencoder.compile(optimizer='adam', loss='binary_crossentropy')

autoencoder.fit(X_train, X_train, epochs=30, batch_size=256, validation_data=(X_test, X_test))

reconstructed = autoencoder.predict(X_test)

plt.figure(figsize=(20, 4))

for i in range(10):

    for img in [X_test[i], reconstructed[i]]:

        plt.subplot(2, 10, i + 1 + (img is reconstructed[i]) * 10)

        plt.imshow(img.reshape(28, 28), cmap='gray')

        plt.axis('off')

plt.show()
```